

exp-mlkem-f203-pke-keygen-k4

FIPS 203 Alg.13 ML-KEM-768 PKE KeyGen 实现方案 (k=4)

ascendc 工程 (预研)

2026-06-26

摘要

本文档描述 **交付前示例** `examples/incubating/exp-mlkem-f203-pke-keygen-k4/` 在 **Atlas A2 / Ascend910B4** 上以 AscendC 实现 FIPS 203 **Algorithm 13** (ML-KEM-768, $K = 4$) **PKE KeyGen**。生产 I/O: `input/` 仅 SEED_D + NTT LUT; `output/` 仅 `ek_PKE(1568B)/dk_PKE(1536B)`。
设备 Launch: 2 次——Launch 1 行 3–15 (`f203_keygen_prep`); Launch 2 行 16–21 (`vec-k4-v2` + 行 21 $\parallel \rho$ 设备融合)。中间 GM 数据 **不落盘**; 无探针门禁 (G0–G4) 概念。验证基线: `ascendc-tests/fix-f203-alg13-device-keygen-k4` (仅对照, 非本目录用户接口)。技术总结: `docs/notes/F203-KeyGen-prep双AIV与SHAKE内嵌技术总结.md`、`docs/notes/F203-KeyGen-prep-Pipe细同步技术总结.md`。

目录

1 数学语义 (Alg.13, $K = 4$, $N = 256$, $q = 3329$)	2
1.1 输入输出	2
1.2 流水线	2
2 编排与 Launch	2
3 AI Core 规模	2
4 数据与组件同步规划	3
4.1 范围与 launch 边界	3
4.2 组件拓扑 (prep)	3
4.3 GM 数据面	3
4.4 PipeBarrier 策略 (生产默认)	4
4.5 双 AIV 与 SHAKE 内嵌	4
4.6 实现映射与 include 链	4
4.7 变更门禁	5
5 验证	5
6 性能 (SIM 910B4 参考, Opt-4+5)	5

1 数学语义 (Alg.13, $K = 4$, $N = 256$, $q = 3329$)

1.1 输入输出

- 输入 SEED_D: uint32, Host 写入 GM; 探针约定 DerandFromSeedD。
- 输出 ek_PKE: 1568 B; dk_PKE: 1536 B。
- 中间 GM: 仅在单次 ascendc_keygen_bbit 进程内驻留; **禁止**写入 input//output/。

1.2 流水线

1. $G(d||k) \rightarrow (\rho, \sigma)$ 。
2. 行 3-7: $16 \times \text{SampleNTT}(\rho) \rightarrow \hat{A}$ 。
3. 行 8-15: $\text{PRF} + \text{CBD}_{\eta_2} \rightarrow \hat{s}$ 。
4. 行 16-20: vec-k4-v2 (NTT、内积、ByteEncode₁₂)。
5. 行 21 (设备 AIV) : $\text{ek_PKE} = \text{ek_polyvec} \parallel \rho$, 与行 16-20 同次 Launch 2 内核内融合 (F203_KEYGEN_EK_PKE)。

2 编排与 Launch

- 用户入口: `bash run.sh -r cpu|sim -v Ascend910B4`; 无 KEYGEN_GATE、无分段门禁。
- **Launch 1**: `f203_keygen_prep` (AIV-only, blockDim=2), 输出 $\hat{A}$ 、 $\hat{s}$ 、 ρ 于 GM。
- **Launch 2**: `mmad_custom` (MIX, F203_KEYGEN_EK_PKE=1), 行 16-20 + 行 21 $\parallel \rho$; 输出 ek_PKE/dk_PKE 于 GM 后仅落盘交付文件。
- **安全**: 进程退出前 Host 释放全部 GM; 不创建 work/ 等持久中间目录。

3 AI Core 规模

段	核类型 / blockDim
Prep	KERNEL_TYPE_AIV_ONLY; aiv=2 默认
Compute	KERNEL_TYPE_MIX_AIC_1_2; aicore=1

内嵌 SHAKE 使用 ProcessInline (见技术总结 §3)。

4 数据与组件同步规划

本节约定 **prep 单内核** (行 3–15) 内 UB/GM 生产者–消费者关系及 PipeBarrier 策略。**compute** (vec-k4-v2, 行 16–20) 为独立 launch, MIX 核内同步不在本节展开; prep 与 compute 之间仅通过 **GM 落盘完成 + Host 顺序 launch** 保证可见性。详实实验记录: `ascendc-tests/fix-f203-alg13-device-keygen-k4/PIPE_SYNC_EVAL.md`; 原理: `docs/notes/F203-KeyGen-prep-Pipe` 细同步技术总结.md。

4.1 范围与 launch 边界

边界	同步机制
prep 内核内 (单 TPipe)	PipeBarrier / 段间 PIPE_ALL
prep block0 $\leftrightarrow$ block1	P-04: 全体末尾 PIPE_ALL (block1 等待 block0 PRF+CBD)
prep $\rightarrow$ compute	无设备侧 barrier ; 同一 Host 进程内 GM 指针传递; 无文件交接

4.2 组件拓扑 (prep)

1. **G 派生** (标量 UB): `BuildRhoSigmaFromSeedD` $\rightarrow \rho, \sigma$ 。
2. **$\hat{A}$** (双 AIV): `BuildAHat16ShardWithUb`, 每核 8 poly; 内嵌 `SHAKE ProcessInline`。
3. **PRF+CBD** (block0 独占): `RunShakePrfBatchUbWithUb` $\rightarrow$ `SamplePolyCbd2Batch8WithUb`。
4. **编排壳**: `f203_keygen_prep_ub.hpp` 持有 P-02/P-03/P-04 段间 barrier。

TPipe UB 复用 (P-02 须 PIPE_ALL): $\hat{A}$ 结束后 `shakeXBuf/shakeLenBuf/shakeStagingBuf/aHatQue/scratch` 交 PRF/CBD 复用; 窄化段间 barrier 曾导致 SIM 风险, **保留 ALL**。

4.3 GM 数据面

GM 张量	生产者	消费者	同步要点
<code>a_hat[4096]</code>	$\hat{A}$ 每 poly DataCopy	compute vec-k4-v2	$\hat{A}$ 循环内 PIPE_ALL (未窄化); 跨 launch 靠 prep 结束
<code>prf_out[1024]</code>	PRF DataCopy	CBD 行 DataCopy	P-03 PIPE_ALL; PRF 自身 PipeAll()
<code>src[2048]</code>	CBD 行 DataCopy	compute	CBD: CopyIn MTE2、Vector 前 V; CopyOut 后 无 barrier (C-04 删减)

4.4 PipeBarrier 策略（生产默认）

ID	组件/文件	上下文	策略
P-02	f203_keygen_pre p_ub.hpp	Â 返回 → PRF	PIPE_ALL
P-03	同上	PRF 结束 → CBD	PIPE_ALL （删减 tick 变差，保留）
P-04	同上	block0 PRF+CBD 后； block1 汇合	PIPE_ALL
C-02	f203_cbd_eta2_u b_io.hpp	GM→UB 后 Vector 读	PIPE_MTE2
C-03	同上	Vector 后 CopyOut 前	PIPE_V
C-04	同上	CopyOut 写 src 后	删减 （下一行 CopyIn 的 MTE2 覆盖）
R-*	f203_se_vector_ prf.hpp	PRF 全段	PipeAll() （Phase 2 窄化 已否决）
A-*	f203_a_hat16_ub .hpp	每 poly SHAKE/rej/GM	PIPE_ALL （Phase 3 窄化已否决）
D/J-*	Alg7 d12/rej	标量 ↔ Vector 链	PIPE_ALL （Phase 4 窄化 SIM FAIL）
S-01	shake_general.h	内嵌 SHAKE	调用点 A-06/R-03 处 ALL; ProcessInline 修 复 block 语义

4.5 双 AIV 与 SHAKE 内嵌

- F203_AHAT16_BLOCK_DIM=2: poly 0-7 / 8-15 分片；GM 不交叠。
- 内嵌 RunKernelShakeGeneralUb **必须**走 ProcessInline (blockIdx_local=0)，禁止在外层 blockDim=2 下用 Process() 分 batch。
- block1 在 PRF 段空转，但 **不得提前 return**；依赖 P-04。

4.6 实现映射与 include 链

f203_keygen_prep_entry.cpp

- > f203_keygen_prep_ub.hpp (P-02/P-03/P-04)
- > f203_a_hat16_ub.hpp (A-*, via AHAT_INC)
- > f203_se_vector_prf.hpp (R-*, via PRESAMPLE_INC)
- > f203_cbd_eta2_ub_io.hpp (C-02/C-03/C-04, via ALG8_INC)
- > shake_general.h / shake_ub_helpers.hpp (S-01, via SHAKE_XOF_INC)

exp 目录 f203_keygen_prep_sync.hpp 集中声明编排层 macro 与规格 ID 对照；子模块策略以 **ascendc-tests** 共享头为准 (cmake **INC* 禁止 *exp fork*)

4.7 变更门禁

1. 修改 barrier 策略须先更新本节表格，再改对应共享头或 f203_keygen_prep_ub.hpp。
2. 验收：G4 CPU + SIM_DIRECT；golden 为准，tick 为辅。
3. 仅 CPU PASS 不得合入删 barrier 类变更。

5 验证

```
cd examples/incubating/exp-mlkem-f203-pke-keygen-k4
bash run.sh -r cpu -v Ascend910B4
bash run.sh -r sim -v Ascend910B4 # -r sim 时 run.sh 自动 export SIM_DIRECT=1
```

6 性能 (SIM 910B4 参考, Opt-4+5)

prep ≈ 447126 (Opt-5 CBD 细同步 + C-04 删减；Opt-4 双 AIV)；compute ≈ 77997 ；合计 ≈ 525123 tick (2026-06-26 SIM 复验)。CBD 经 ascendc-tests/fix-f203-alg8-cbd-eta2-k4/f203_{cbdetasubio.hpp} MTE2/V